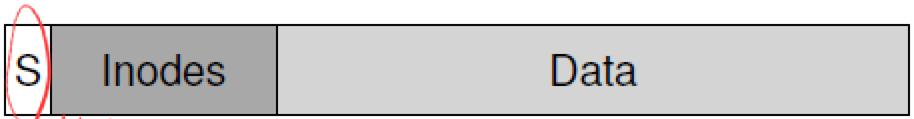
Locality and The Fast File System

First File System

- "old UNIX file system" by Ken Thompson



super block

- simple
- supported files and the directory hierarchy

Kirk McKusick

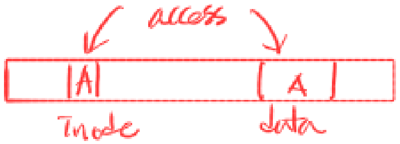
- The problem: performance was terrible.
  - Performance started off bad and got worse over time, to the point where the file system was delivering only 2% of overall disk bandwidth! [Kirk McKusick '84]

100MB/s → 2MB/s

performance가 나쁜 원인

- Treated the disk like it was a random-access memory; data was spread and thus had real and expensive positioning costs. → seek time
  - For example, the data blocks of a file were often very far away from its inode

Inode와 data block이 멀리 떨어짐

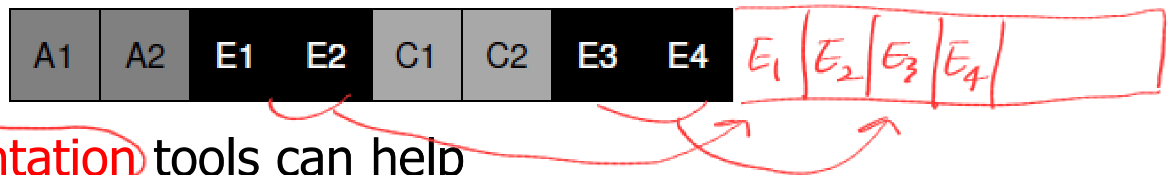


서로 멀리 떨어지면 seek time↑

First File System

- The file system would end up getting quite fragmented
  - free space was not carefully managed.

    disk에서 access하기 위해선 앞뒤로 왔다갔다 해야 됨
  - a logically contiguous file would be accessed by going back and forth across the disk, thus reducing performance dramatically.



  - defragmentation tools can help
    - reorganize on-disk data to place files contiguously and make free space for one or a few contiguous regions, moving data around and then rewriting inodes and such to reflect the changes.

high overhead

요즘은 4KB사용

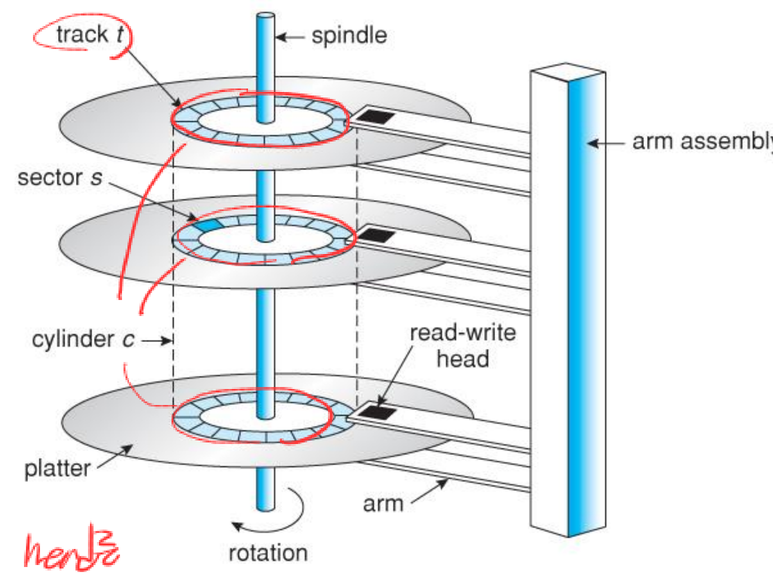
- One other problem: the original block size was too small (512 bytes).
  - Smaller blocks minimized internal fragmentation (waste within the block),
  - but bad for transfer as each block might require a positioning overhead to reach it.

small block이 internal fragmentation에는 좋지만 data transfer에는 비효율적임

FFS: Disk Awareness Is The Solution

hardware structure를 aware

- Fast File System (FFS)
  - Design the file system structures and allocation policies to be "disk aware" and thus improve performance
  - Same APIs, but advanced internal implementation
- FFS divides the disk into a number of cylinder groups.
  - A single cylinder is a set of tracks on different surfaces of a hard drive that are the same distance from the center of the drive
  - aggregates each N consecutive cylinders into group



cylinder group내 data는 head를 움직이지 않아 seek time으로 인한 overhead X

FFS: Disk Awareness Is The Solution

disk 내 physical한 구조에 대해 알 수 없음

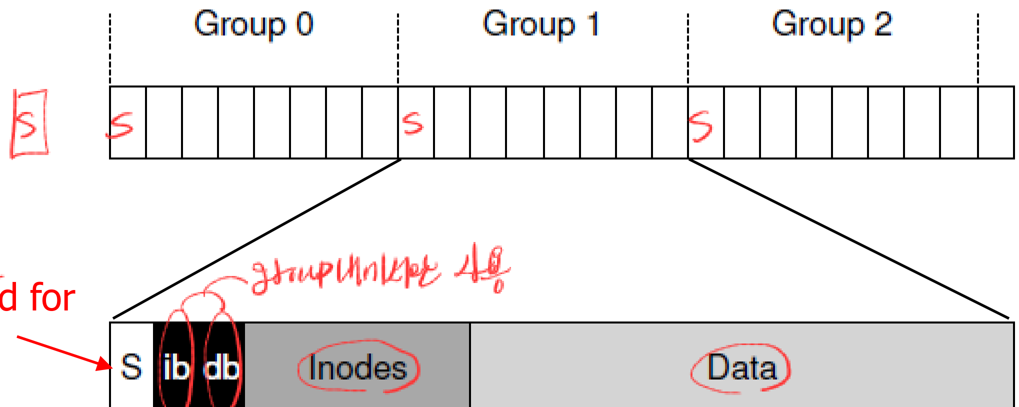
- Modern drives do not export enough information for the file system to truly understand whether a particular cylinder is in use
- Modern file systems (such as Linux ext2, ext3, and ext4) instead organize the drive into block groups

  때문에 cylinder group 대신 block group이라는 logical space를 사용

  - A block group is just a consecutive portion of the disk's address space.

    physical하게 가까운 것이라고 가정

- By placing two files within the same group, FFS can ensure that accessing one after the other will not result in long seeks across the disk.



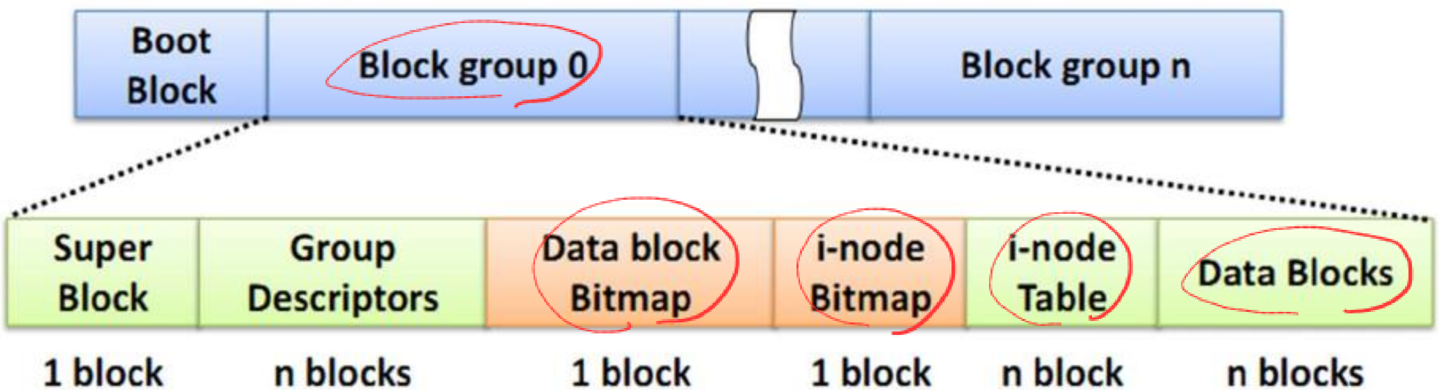
각 group마다 superblock을 둠

Duplicated for reliability

per-group inode bitmap (ib) and data bitmap (db)

Linux File System

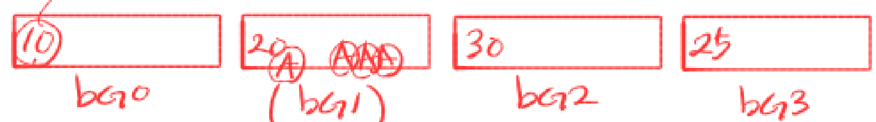
- **ext2/ext3/ext4**
  - Block group
    - Kernel tries to keep the data blocks belonging to a file in the same block group to reduce file fragmentation
    - All the block groups have the same size and are stored sequentially



Policies: How To Allocate Files and Directories

- Keep related stuff together (keep unrelated stuff far apart)!!!
- FFS has to decide what is "related" and place it within the same block group
- Simple placement heuristics

가장 적은 directory이므로 새 directory가 여기에 할당됨



inode와 data를 같은 group으로

  - Placement of a directory

directory

    - find the cylinder group with
    - a low number of allocated directories (to balance directories across groups)
    - a high number of free inodes (to subsequently be able to allocate a bunch of files)
    - other heuristics could be used here (e.g., a high number of free data blocks)
  - File

file

    - allocate the data blocks of a file in the same group as its inode
    - places all files that are in the same directory in the cylinder group of the directory they are in.
    - if a user creates four files, /a/b, /a/c, /a/d, and b/f, FFS would try to place the first three in the same group and the fourth in some other group.

same parent directory a
(/a의 block group에 넣음)

다른
group으로

Policies: How To Allocate Files and Directories

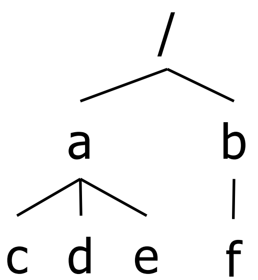
FFS policies

same block group

(1) The data blocks of each file are near each file's inode

(2) The files in the same directory are near one another

⇒ HDD → seek



directory hierarchy

SSD도 Random access 보다
sequential access의 성능이 더 좋다.
(locality에 기반한 allocation이 여전히 성능↑)

Are these policies still helpful for SSD-based systems?

flash ↳ yes

directory entry

a, c, d, e를 순차적으로 access할 경우 ①~⑧

```
group inodes      data
    0 /---------  /---------
    1 acde------  accddee---
    2 bf--------  bff-------
    3 ----------  ----------
    4 ----------  ----------
    5 ----------  ----------
    6 ----------  ----------
    7 ----------  ----------
  ...
```

(1) & (2) (namespace-based locality)

block group에 흩어짐

```
group inodes      data
    0 /---------  /---------
    1 a---------  a---------
    2 b---------  b---------
    3 c---------  cc--------
    4 d---------  dd--------
    5 e---------  ee--------
    6 f---------  ff--------
    7 ----------  ----------
  ...
```

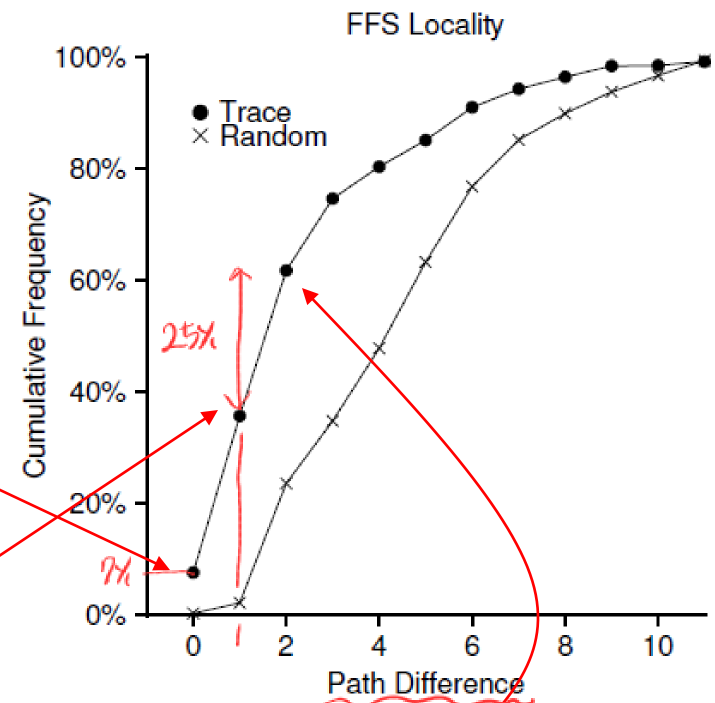
이렇게 disk access overhead를 줄여주긴 함

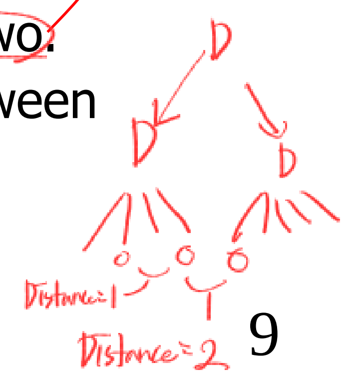
Only (1)만 지원할 경우
같은 directory의 file이 모여지 않음

Measuring File Locality

- Let's see if indeed there is namespace locality
- Path difference
  - how "far away" file accesses were from one another in the directory tree
  - Same file accesses: 0
  - Different file in the same directory: 1
- SEER traces
  - about 7% of file accesses were to the file that was opened previously
  - nearly 40% of file accesses were to either the same file or to one in the same directory (distance ≤ 1)
  - 25% of file accesses were to files that had a distance of two.
    - a set of related directories and consistently jumps between them (e.g., /proj/src/foo.c and proj/obj/foo.o)
    - FFS does not capture this type of locality

↳ distance: 2





The Large-File Exception

- Problem of FFS policies
  - If a large file would entirely fill the block group it is first placed within, it prevents subsequent "related" files from being placed within this block group, and thus may hurt file-access locality.

```
group inodes     data
    0 /a-bc----- /aaaaaaaaa aaaaaaaaaa aaaaaaaaaa a-bbbbbbbb
    1 ---------- -bbb-ccccc ---------- ---------- ----------
    2 ---------- ---------- ---------- ---------- ----------
  ...
```

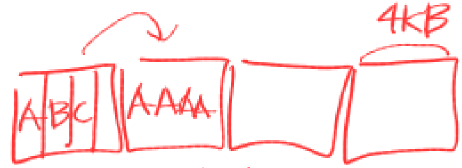
locality를 해침

- Large-File Exception
  - The first twelve direct blocks are allocated into the first block group (48KB)
  - The next "large" chunk of the file in another block group.
    - Each subsequent indirect block, and all the blocks it pointed to. (1024 blocks = 4 MB)
  - If the chunk size is large enough, little seeking overhead

```
group inodes     data
    0 /a-------- /aaaaa---- ---------- ---------- ----------
    1 ---------- aaaaa----- ---------- ---------- ----------
    2 ---------- aaaaa----- ---------- ---------- ----------
    3 ---------- aaaaa----- ---------- ---------- ----------
    4 ---------- aaaaa----- ---------- ---------- ----------
    5 ---------- aaaaa----- ---------- ---------- ----------
    6 ---------- ---------- ---------- ---------- ----------
  ...
```

A Few Other Things About FFS

- Small files < 4KB
  - Internal fragmentation
  - sub-block allocation (512B)
  - if the file grows and requires a full 4KB of data, copy the sub-blocks into a 4KB block, and free the sub-blocks for future use.
  - To avoid a lot of extra I/O to perform the copy, the libc library would buffer writes and then issue them in 4KB chunks to the file system.



A가 커질 경우 copy

- Made the system more usable
  - Long file name (only 8 characters in the traditional fixed-size approach)
  - Symbolic link
  - Atomic rename()
- Many recent file systems take cues from FFS

EXT4
거의 유사

Homework

- Homework in Chap 41 (FFS)

Quiz

같은 cylinder group → 헤드를 움직일 필요X

1. FFS divides the disk into cylinder groups to improve locality. (O)
2. In FFS, each cylinder group contains its own inode bitmap and data bitmap. (O)
3. In FFS, the data blocks of a file are placed ~~far from~~ the file's inode. (X)
4. FFS tries to place files from the same directory into the same cylinder group. (O)

   너무 크면 → 분산시킴
5. FFS places all blocks of a file in the same group ~~regardless of size~~. (X)
6. FFS provides atomic rename operation. (O)
7. FFS tries to place related files in ~~different~~ block groups for parallel access. (X)
8. For small files (<4KB), FFS uses sub-block allocation to reduce internal fragmentation. (O)
9. Modern Linux file systems such as ext2/3/4 use ~~cylinder groups~~, just like FFS did. (X)

   data group
10. Since SSDs do not have seek latency, block groups are ~~not required~~ on SSDs. (X)

    SSD에서도 유효함